NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Week 8 Laboratory on Stacks

Balanced Parentheses and Arithmetic Evaluator in C++

Name: Octaviano, Brittany Ashley M. Octaviano
Section: 2.4 BSIT
Date: November 4, 2026

Repository: https://github.com/OCTAVIANO-BRITTANY-M/CC105_Week8_Stack_Laboratory

Part 1 – Build and Verify the Stack

Stack Implementation

The application uses a reusable template Stack<T> implemented using a singly linked list. It supports push, pop, peek, isEmpty, size, and clear operations. Copy construction and copy assignment are disabled to prevent unsafe shallow copying. The destructor releases the remaining nodes.

Stack Test Results

Input / Operation	Expected Output	Actual Output	Result
Pop from an empty stack	Returns false; no crash	Returns false; no crash	PASS
Push 10, then push 20; peek	Top is 20; size is 2	Top is 20; size is 2	PASS
Push 30	Stack from bottom to top: 10, 20, 30	10, 20, 30	PASS
Pop three times	Values removed: 30, 20, 10	30, 20, 10	PASS
Peek from an empty stack	Returns false; no crash	Returns false; no crash	PASS
Display stack bottom to top	Values displayed in insertion order	Values displayed from bottom to top	PASS

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Part 2 – Check Balanced Brackets

The program reads the full expression using getline(). Opening brackets are pushed onto the stack. When a closing bracket appears, the program checks whether it matches the most recently opened bracket.

Bracket Test Results

Input	Expected Output	Actual Output	Result
{[2 + (3 * 4)]}	Balanced brackets	Balanced brackets	PASS
([)]	Mismatched brackets	Mismatched brackets	PASS
(2 + 3	Unclosed opening bracket	Unclosed opening bracket	PASS
2 + 3)	Unmatched closing bracket	Unmatched closing bracket	PASS

Checkpoint Answer

The expression {[2 + (3 * 4)]} is balanced because every opening bracket has a matching closing bracket in the correct order. On the other hand, ([)] is mismatched because the closing parenthesis) appears while the most recent opening bracket is [, which does not match.

Counting opening and closing brackets alone cannot detect every error because it only checks whether the number of opening and closing brackets is equal. It does not check their order or types. A stack is needed to ensure that each closing bracket matches the most recently opened bracket.

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Part 3 – Tokenize and Convert to Postfix

The program scans the expression from left to right, combines consecutive digits into integer tokens, skips whitespace, and rejects unsupported characters. It uses operator precedence to convert infix expressions into postfix notation.

Postfix Conversion Test Results

Input	Expected Output	Actual Output	Result
(12 + 3) * 2	12 3 + 2 *	12 3 + 2 *	PASS
8 + 2 * 5	8 2 5 * +	8 2 5 * +	PASS
20 / 5 / 2	20 5 / 2 /	20 5 / 2 /	PASS
10 - 3 - 2	10 3 - 2 -	10 3 - 2 -	PASS
{[2 + 3] * (4 + 1)}	2 3 + 4 1 + *	2 3 + 4 1 + *	PASS
2 + * 3	Invalid operator sequence	Invalid operator sequence	PASS
2 3 + 4	Missing operator	Missing operator	PASS
2(3 + 4)	Implicit multiplication rejected	Implicit multiplication rejected	PASS
()	Invalid empty group	Invalid empty group	PASS
Empty input	Invalid expression	Invalid expression	PASS
-5 + 2	Unsupported unary sign	Unsupported unary sign	PASS
2.5 + 1	Unsupported decimal	Unsupported decimal	PASS
2 + a	Unsupported character	Unsupported character	PASS
2 +	Missing operand	Missing operand	PASS

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Part 4 – Evaluate Postfix

The program pushes numeric tokens onto a Stack<double>. For each operator, it pops the right operand first and the left operand second. It then operates and pushes the result.

Postfix Evaluation Test Results

Input	Expected Output	Actual Output	Result
10 2 -	Result: 8	Result: 8	PASS
7 2 /	Result: 3.5	Result: 3.5	PASS
0	Result: 0	Result: 0	PASS
5 / (3 - 3)	Division by zero	Division by zero	PASS
Operator with fewer than two operands	Invalid postfix expression	Invalid postfix expression	PASS
Postfix expression with extra operands	Invalid expression; exactly one value required	Invalid expression; exactly one value required	PASS
Empty postfix stack	Invalid expression; no result available	Invalid expression; no result available	PASS

Checkpoint Answer

The postfix expression 10 2 - gives the result **8**. During evaluation, the stack first receives 10 and then 2. When the subtraction operator is encountered, 2 is popped first as the right operand, followed by 10 as the left operand. The operation is therefore 10 - 2 = 8.

The order matters because subtraction and division are not commutative. Reversing the operands would produce a different result. Evaluation must also stop immediately when an error occurs to avoid using invalid or incomplete values.

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Part 5 – Integrate and Test the Application

The application provides a repeating menu with the following options:

1. Check brackets
2. Convert and evaluate expression
3. Run stack demonstration
4. Exit

Input / Action	Expected Output	Actual Output	Result
Select 1	Bracket-checking feature starts	Bracket-checking feature starts	PASS
Select 2	Expression conversion and evaluation starts	Expression conversion and evaluation starts	PASS
Select 3	Stack demonstration runs	Stack demonstration runs	PASS
Enter invalid menu input	Error message appears and menu is displayed again	Error message appears and menu is displayed again	PASS
Select 0	Program exits	Program exits	PASS
Enter a valid expression	Shows input, bracket status, postfix expression, and result	Shows input, bracket status, postfix expression, and result	PASS
Enter an invalid expression	Shows an error and returns to the menu	Shows an error and returns to the menu	PASS

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Menu and Integration Test ResultsRequired Valid Expression Tests

Input	Expected Output	Actual Output	Result
(12 + 3) * 2	12 3 + 2 *; result 30	12 3 + 2 *; result 30	PASS
8 + 2 * 5	8 2 5 * +; result 18	8 2 5 * +; result 18	PASS
20 / 5 / 2	20 5 / 2 /; result 2	20 5 / 2 /; result 2	PASS
10 - 3 - 2	10 3 - 2 -; result 5	10 3 - 2 -; result 5	PASS
{[2 + 3] * (4 + 1)}	2 3 + 4 1 + *; result 25	2 3 + 4 1 + *; result 25	PASS
7 / 2	7 2 /; result 3.5	7 2 /; result 3.5	PASS
0	0; result 0	0; result 0	PASS

Required Invalid Expression Tests

Input	Expected Output	Actual Output	Result
([)]	Mismatched brackets	Mismatched brackets	PASS
(2 + 3	Unclosed opening bracket	Unclosed opening bracket	PASS
2 + 3)	Unmatched closing bracket	Unmatched closing bracket	PASS
5 / (3 - 3)	Division by zero	Division by zero	PASS
2 + * 3	Invalid operator sequence	Invalid operator sequence	PASS
2 3 + 4	Missing operator	Missing operator	PASS
2(3 + 4)	Implicit multiplication rejected	Implicit multiplication rejected	PASS
()	Invalid expression	Invalid expression	PASS
Empty input	Invalid expression	Invalid expression	PASS

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Input	Expected Output	Actual Output	Result
-5 + 2	Unsupported unary sign	Unsupported unary sign	PASS
2.5 + 1	Unsupported decimal	Unsupported decimal	PASS
2 + a	Unsupported character	Unsupported character	PASS
2 +	Missing operand	Missing operand	PASS

Original Test Cases

Input	Expected Output	Actual Output	Result
{[(2 + 3) * (4 + 1)]}	2 3 + 4 1 + *; result 25	2 3 + 4 1 + *; result 25	PASS
1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10	1 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 +; result 55	1 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 +; result 55	PASS
((2 + 3) *)	Invalid expression due to missing operand	Invalid expression due to missing operand	PASS

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Part 6 – Compare Stack Behavior

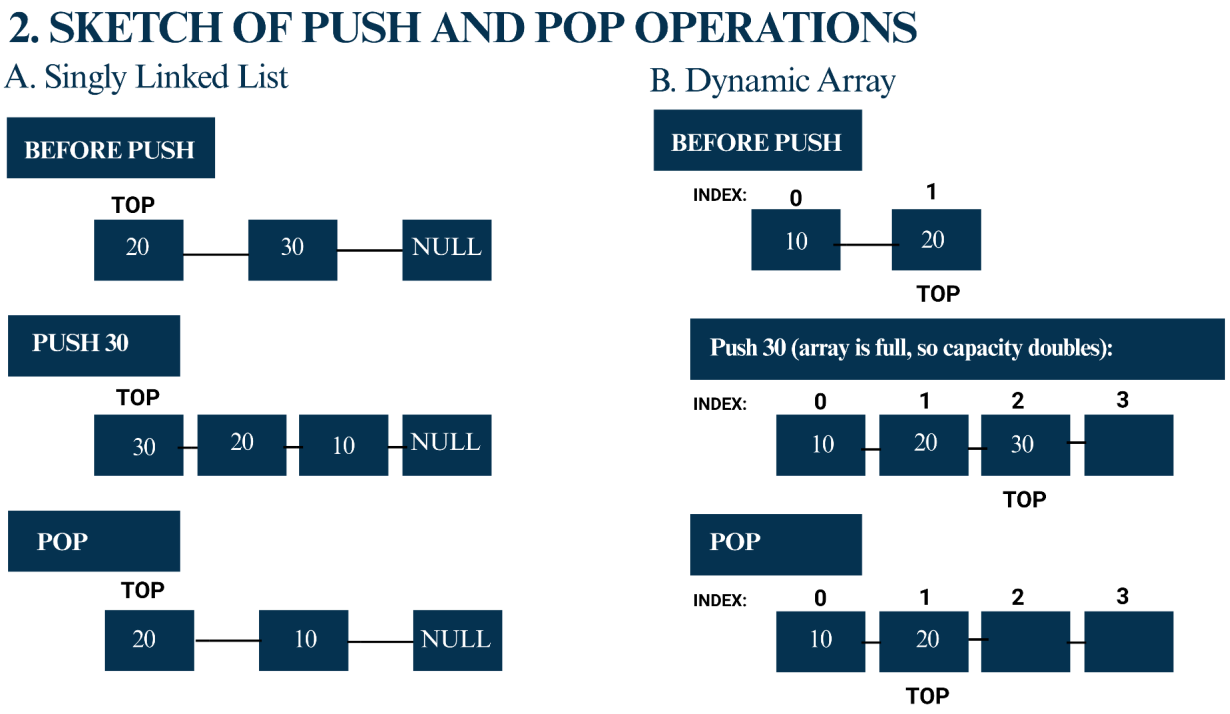
1. Comparison of Stack Implementations

My program uses a singly linked list as its stack implementation. The other possible implementation is a dynamically resized array.

In a linked-list stack, pushing an element creates a new node and places it at the top. Popping removes the top node and releases its memory.

In a dynamic-array stack, pushing an element places it in the next available index. If the array is full, its capacity doubles and the existing elements are copied into a larger array. Popping removes the last element by decreasing the stack size.

2. Sketch of Push and Pop Operations



NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

3. Time Complexity Comparison

Stack Operation	Linked List	Dynamic Array
Push	O(1)	Amortized O(1)
Push during resizing	O(1)	O(n)
Pop	O(1)	O(1)
Peek	O(1)	O(1)
Check if empty	O(1)	O(1)

Linked-list push and pop are O(1) because they only update the top pointer and do not require shifting other elements. Dynamic-array push is amortized O(1) because most insertions take constant time, but it becomes O(n) when the array is full and its elements must be
copied into a larger array.

4. Complexity of Expression Algorithms

Algorithm	Time Complexity	Auxiliary Space
Balanced Bracket Checking	O(n)	O(n)
Infix-to-Postfix Conversion	O(n)	O(n)
Postfix Evaluation	O(n)	O(n)

Explanation:

- Balanced Bracket Checking: The program scans each character once. Opening brackets are pushed onto the stack, while closing brackets are checked and popped. Therefore, the time complexity is O(n), and the stack may store up to O(n) brackets.
- Infix-to-Postfix Conversion: The program scans the input once. Each token is added to the output or pushed and popped from the operator stack. Each operator is pushed and popped at most once, making the time complexity O(n). The output and operator stack may require O(n) auxiliary space.
- Postfix Evaluation: The program scans each token once. Numbers are pushed onto the stack, and operators pop two operands and push the result. Therefore, the time complexity is O(n), and the value stack may require O(n) space.

These complexity estimates assume fixed-width token arithmetic, meaning each token operation takes constant time. Here, n represents the input length.

5. Conclusion

Both stack implementations follow the Last-In, First-Out principle. A linked list provides constant-time push and pop operations, while a dynamic array provides amortized constant-time push operations but may require resizing. The three expression algorithms have O(n) time complexity and O(n) auxiliary space because they process the input once and use stacks to store temporary values.

Conversion Trace, Postfix Evaluation Trace, and Reflection Questions

A. Original Expression

The original expression used for the conversion and evaluation traces is:

```
{[(6 + 2) * (9 - 4)] / 2}
```

Expected Postfix Expression:

```
6 2 + 9 4 - * 2 /
```

Expected Result: 20

B. Infix-to-Postfix Conversion Trace

The expression is scanned from left to right. Numbers are added directly to the postfix output, while operators and opening brackets are stored in the stack until they can be processed.

Step	Scanned Token	Operator Stack (Bottom to Top)	Postfix Output
1	{	{	
2	[	{ [	
3	(	{ [(	
4	6	{ [(	6
5	+	{ [(+	6
6	2	{ [(+	6 2
7	)	{ [	6 2 +

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Step	Scanned Token	Operator Stack (Bottom to Top)	Postfix Output
8	*	{ [*	6 2 +
9	(	{ [* (	6 2 +
10	9	{ [* (	6 2 + 9
11	-	{ [* (-	6 2 + 9
12	4	{ [* (-	6 2 + 9 4
13	)	{ [*	6 2 + 9 4 -
14	]	{	6 2 + 9 4 - *
15	/	{ /	6 2 + 9 4 - *
16	2	{ /	6 2 + 9 4 - * 2
17	}	Empty	6 2 + 9 4 - * 2 /

Final Postfix Expression:

6 2 + 9 4 - * 2 /

C. Postfix Evaluation Trace

The postfix expression is evaluated from left to right using a stack. Numbers are pushed onto the stack. When an operator is encountered, the two top values are popped, the operation is performed, and the result is pushed back.

Step	Token	Operation Performed	Stack (Bottom to Top)
1	6	Push 6	6
2	2	Push 2	6, 2

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Step	Token	Operation Performed	Stack (Bottom to Top)
3	`+`	Pop 2 and 6. Calculate 6 + 2 = 8. Push 8.	`8`
4	`9`	Push 9	`8, 9`
5	`4`	Push 4	`8, 9, 4`
6	`-`	Pop 4 and 9. Calculate 9 - 4 = 5. Push 5.	`8, 5`
7	`*`	Pop 5 and 8. Calculate 8 × 5 = 40. Push 40.	`40`
8	`2`	Push 2	`40, 2`
9	`/`	Pop 2 and 40. Calculate 40 / 2 = 20. Push 20.	`20`

Final Stack Value: 20

Final Evaluation Result: 20

The evaluation is successful because only one value remains in the stack after all postfix tokens have been processed.

NATIONAL TEACHERS COLLEGE
CC105 – Data Structures with Lab

Reflection Questions

1. Why does the Last-In, First-Out (LIFO) principle correctly match nested brackets?

The LIFO principle works because the last opening bracket must be the first one to close. A stack keeps track of the most recently opened bracket, making it easier to check if the closing brackets are in the correct order.

2. Why must the evaluator pop the right operand before the left operand?

The evaluator must pop the right operand first, followed by the left operand, to maintain the correct order of the operation. This is important for subtraction and division because reversing the operands produces a different answer.

3. What is the difference between balanced brackets and a valid arithmetic expression?

Balanced brackets mean that all opening and closing brackets match and are arranged in the correct order. A valid arithmetic expression must also follow the correct arrangement of numbers and operators. An expression can have balanced brackets but still contain an invalid mathematical operation or syntax.

4. How is a stack similar to the function call stack? Which nested call returns first?

Both use the Last-In, First-Out principle. In a function call stack, the most recently called function must finish and return first before the earlier function continues. This is similar to how a stack processes nested brackets.

5. What memory does your stack implementation allocate and release? Explain how memory is managed.

My implementation uses a singly linked list. Every time an element is pushed, a new node is created using `new`. When an element is popped, its memory is released using `delete`. The destructor also removes the remaining nodes when the stack is no longer needed. This helps prevent memory leaks.